

Auditoría y Plan de Evolución — de AppVentas a Empresa IA

Documento para aprobación. **No se implementa nada hasta que apruebes.**

Fecha: 2026-07-05 · Proyecto: appventas (Pava Negra / pavanegra.com.ar)

Objetivo: convertir el proyecto en una plataforma **configurable, multi-fabricante y multi-catálogo**, B2B/B2C, para vender productos personalizados (mates, bombillas, materas, cuchillos, tablas, termos, llaveros, pulseras, regalos corporativos, merchandising... cualquier producto) con un equipo de **agentes de IA** que comparten memoria, herramientas y reglas, usando IA como **último recurso**.

0. Resumen ejecutivo (léelo aunque no leas el resto)

Dónde estás hoy: tenés una base sólida y poco común para un negocio de este tamaño. Ya existe una tienda funcional (Next.js 16 + Prisma + Postgres en Vercel), un scraper de competencia multi-plataforma, un prospector B2B por provincia, WhatsApp conectado punta a punta, y — lo más valioso — una **plataforma de 7 capas de "Empresa IA"** ya construida: proveedor de IA multi-modelo, registro de herramientas, servicios de negocio deterministas, motor de memoria, motor de agentes (7 agentes), cola de aprobaciones y agendado por cron.

El problema real no es técnico, es de foco: construiste mucha capacidad y poca de esa capacidad está *conectada a plata*. El scraper, los agentes y la memoria funcionan, pero el negocio hoy tiene poco tráfico, así que la IA no tiene de qué aprender y varios agentes corren "en el vacío". Además hay **dos capas de datos** conviviendo (modelos Prisma en inglés + tablas SQL crudas en español) que es la principal deuda técnica y el mayor riesgo a futuro.

Mi recomendación (diferente a "sumar más agentes"): NO agregar agentes nuevos todavía. Primero **consolidar el dominio de datos** y **hacer configurable el catálogo/fabricante** (que es lo que te desbloquea vender cualquier producto de cualquier proveedor). Recién después, activar de a uno los agentes que tocan plata: **Cotizador** (presupuestos B2B automáticos) e **Inteligencia Comercial** (posición vs. competencia). Taller/BOM y Compras son valiosos pero recién rinden cuando tengas volumen de producción real.

En una frase: *menos agentes nuevos, más configurabilidad y datos limpios; encender los agentes por orden de retorno, no todos a la vez.*

1. Auditoría completa del estado actual

1.1 Stack y plataforma

- **Framework:** Next.js 16.2.9 (App Router, Turbopack). Ojo: esta versión tiene breaking changes; los `params` de rutas dinámicas son `Promise` y hay que `await`. Ya está documentado en AGENTS.md.
- **Datos:** Prisma + PostgreSQL (Neon).
- **Hosting:** Vercel plan Hobby → límite duro de **60s por función**. Ya nos mordió (prospector con `maxDuration=90` rompió el deploy). Es una restricción real de diseño, no un detalle.

- **IA:** capa de abstracción propia (Anthropic SDK + compatible OpenAI vía fetch → OpenAI / OpenRouter / Ollama / LM Studio). Hoy activo: **OpenAI GPT-4o**, ~US\$0.0005 por corrida de agente.

1.2 Las 7 capas de "Empresa IA" (ya existen y funcionan)

- **Proveedor de IA** (`src/lib/ai/*`): multi-proveedor, config en `catalog_config` , costos estimados, claves enmascaradas. ✓
- **Herramientas** (`src/lib/tools/*`): 17 tools con contrato `{name, sideEffect: read|write, input(zod), run}` y validación. ✓
- **Servicios de negocio deterministas** (`src/lib/services/*`): productos, inteligencia (estadísticas con mediana + filtro de outliers), prospectos, presupuesto, pricing (piso de margen 15%), finanzas, whatsapp, calendario, campañas. ✓ **Esta es la joya:** hace trabajo sin gastar tokens.
- **Memoria** (`src/lib/memory/*`): namespaces, confianza, hits, caché de respuestas de IA. ✓ pero **sub-utilizada** (poco tráfico = poco que recordar).
- **Agentes** (`src/lib/agents/*`): 7 agentes (ceo, comercial, compras, finanzas, marketing, whatsapp, calendario) con flujo memoria → reglas → datos → IA → experiencia, telemetría y modos de autonomía (manual/asistido/autónomo). ✓
- **Aprobaciones** (`action_queue`): las acciones de escritura se **proponen** y esperan tu OK (excepto en modo autónomo). ✓
- **Agendado** (`vercel.json` crons): briefing diario, corrida de agentes, carritos abandonados, suscripciones. ✓

1.3 Funcionalidades de negocio que ya tenés

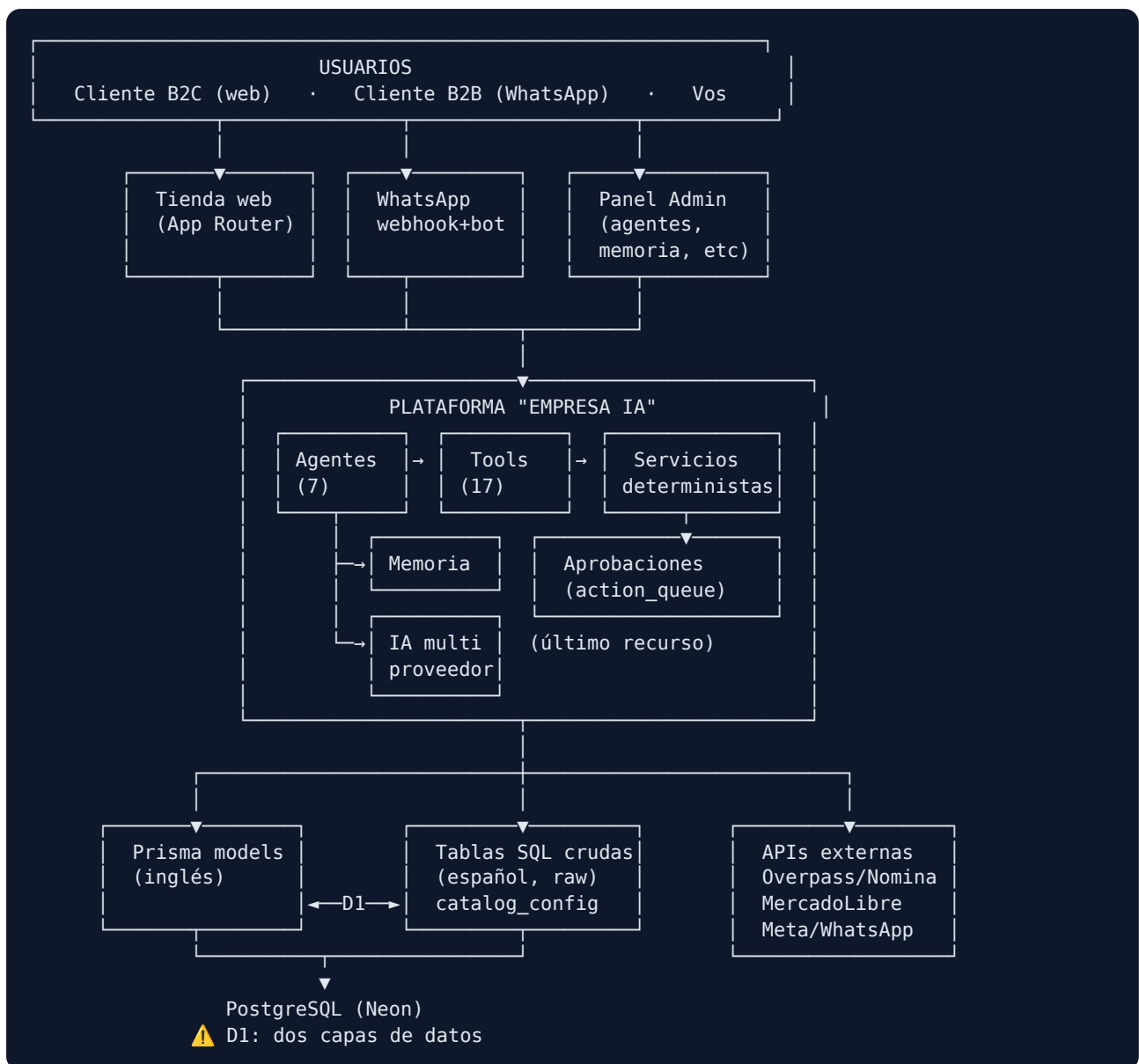
- Tienda e-commerce funcional (catálogo, precios, combos).
- **Scraper de competencia** multi-plataforma (Tiendanube, Shopify, WooCommerce + sniffing + MercadoLibre por HTML). Frágil por naturaleza (depende del HTML ajeno), pero operativo.
- **Prospector B2B** por provincia/país vía OpenStreetMap Overpass + geocoding Nominatim, con Instagram/Facebook, filtros y matching sin acentos.
- **WhatsApp Business** punta a punta (webhook, bot de reglas, normalización de números Argentina, respuestas cálidas).
- **Cross-reference competencia ↔ catálogo** ("Mi posición").
- **Empleado Virtual** fases 1-4: briefing diario, sugeridor de precios con "Aplicar", generador de campañas Meta, mensajes de outreach B2B.
- Manuales PDF en `docs/` .

1.4 Deuda técnica y puntos débiles (lo que hay que mirar de frente)

#	Problema	Severidad	Por qué importa
D1	Dos capas de datos: modelos Prisma (inglés) + tablas SQL crudas (español) vía <code>\$queryRawUnsafe</code> .	● Alta	Duplica la verdad, invita a inconsistencias, y <code>\$queryRawUnsafe</code> esquiva el tipado de Prisma. Es la deuda #1.
D2	Uso de <code>\$executeRawUnsafe</code> / <code>\$queryRawUnsafe</code> con <code>CREATE TABLE IF NOT EXISTS</code> en caliente.	● Media	No hay migraciones versionadas para media plataforma. Difícil de auditar y revertir.
D3	Scraper dependiente de HTML ajeno.	● Media	Se rompe cuando cambian los sitios. Ya pasó varias veces (403/404/406/422).

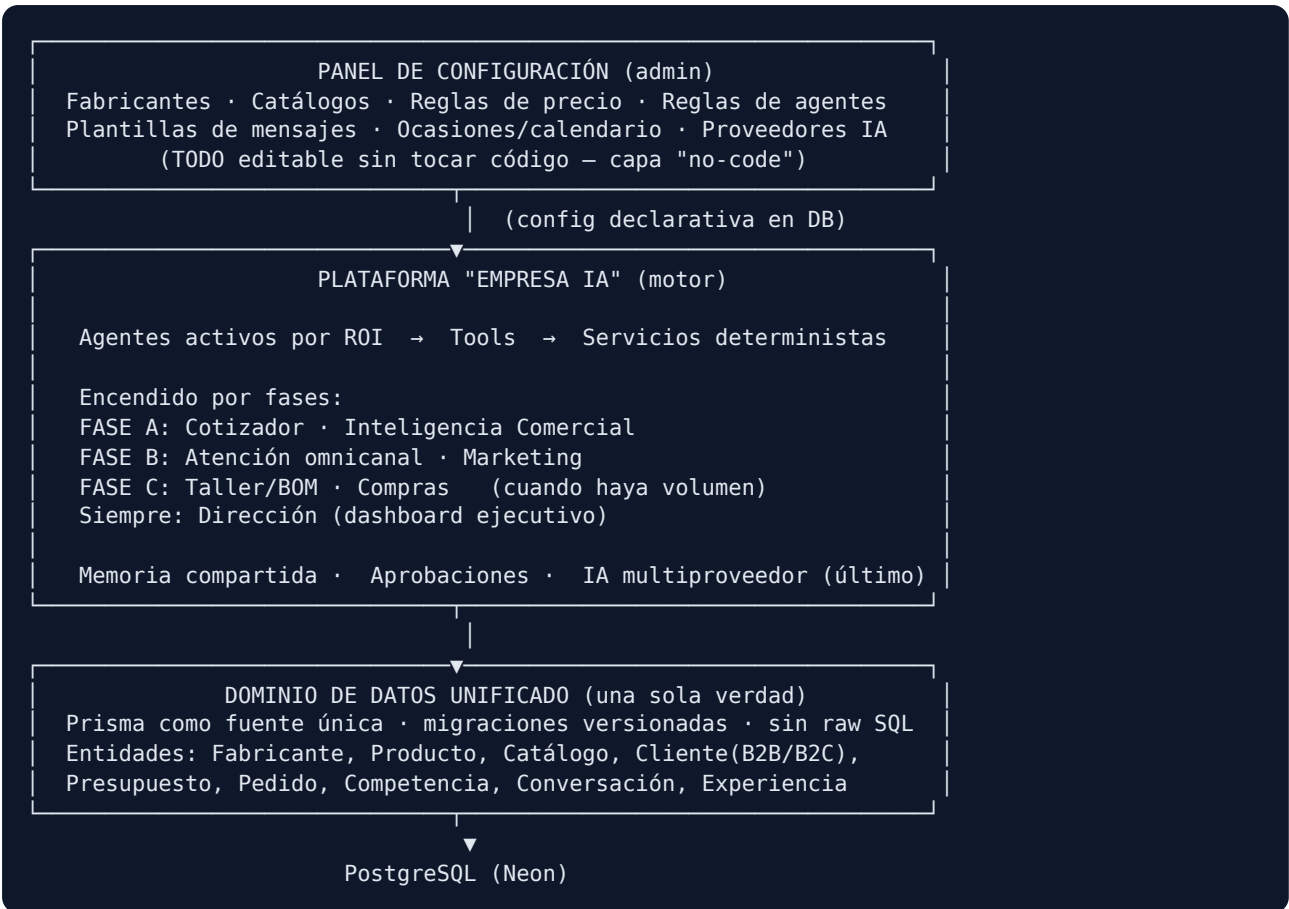
#	Problema	Severidad	Por qué importa
D4	Agentes corriendo sin datos (poco tráfico).	● Media-baja	Gastan tokens sin aprender. Hoy la memoria no se llena.
D5	Catálogo/fabricante no configurable todavía.	● Alta	Es exactamente lo que bloquea "vender cualquier producto de cualquier proveedor".
D6	Límite 60s de Vercel Hobby.	● Baja	Techo para scraping/prospección masiva y para agentes pesados.
D7	Seguridad: ya se hizo un hardening (validación de precio server-side, webhooks/crons fail-closed, token admin firmado + rate limit).	● Ok	Mantener; revisar que todo endpoint nuevo herede el patrón.

2. Diagrama de arquitectura ACTUAL



3. Diagrama de arquitectura PROPUESTA

Idea central: una **capa de configuración de negocio** por encima de todo (multi-fabricante, multi-catálogo, reglas editables desde el admin **sin tocar código**) y un **dominio de datos unificado** debajo. Los agentes no cambian de motor; cambian de "combustible" (datos limpios y configuración).



Qué cambia respecto a hoy: (a) desaparece la doble capa de datos (D1/D2); (b) aparece la capa de configuración no-code que te deja dar de alta un fabricante o un catálogo nuevo desde el admin; (c) los agentes se **encienden por orden de retorno**, no todos juntos.

4. Roadmap por fases

Fase 0 — Fundaciones (habilita todo lo demás)

- **F0.1 Unificar dominio de datos** (resolver D1/D2): migrar las tablas crudas a modelos Prisma con migraciones versionadas. Sin esto, cada agente nuevo agranda el problema.
- **F0.2 Modelo de Fabricante y Catálogo configurable** (resolver D5): entidades + pantallas de admin para dar de alta fabricantes, catálogos y reglas de precio por fabricante.

Fase A — Agentes que tocan plata (encender de a uno)

- **A.1 Cotizador:** presupuestos B2B automáticos a partir de catálogo + reglas (determinista; IA solo para redactar el mensaje). **Mayor ROI.**
- **A.2 Inteligencia Comercial:** consolidar scraper + "Mi posición" en un agente con alertas de precio.

Fase B — Captación y atención

- **B.1 Atención omnicanal:** unificar WhatsApp (+ luego Instagram/Meta) en una sola bandeja con el agente asistido.
- **B.2 Marketing:** campañas por ocasión (ya existe base) + calendario con anticipación (Día de la Madre, etc.).

Fase C — Producción (solo con volumen real)

- **C.1 Taller/BOM:** lista de materiales y costeo de producción.
- **C.2 Compras:** reposición de insumos según BOM y stock.

Continuo

- **Dirección:** dashboard ejecutivo que resume todo (evolución del briefing actual).
- **Seguridad:** mantener el patrón fail-closed en cada endpoint nuevo.

5. Riesgos

Riesgo	Prob.	Impacto	Mitigación
Migración de datos (F0.1) rompe módulos vivos	Media	Alto	Migrar por entidad, con doble escritura temporal y verificación; no "big bang".
Scraper se sigue rompiendo	Alta	Medio	Tratarlo como "best-effort", con fallback y alertas; no depender de él para decisiones críticas.
Agentes gastan tokens sin retorno	Media	Bajo	Encender por fase; IA como último recurso; caché de memoria; modo manual/asistido por defecto.
Límite 60s Vercel Hobby	Media	Medio	Trabajos largos por lotes/cron; evaluar plan Pro solo si hace falta.
Sobre-ingeniería (más agentes que ventas)	Alta	Alto	Este documento: encender por ROI, no por entusiasmo.
Dependencia de un solo proveedor IA	Baja	Bajo	Ya mitigado: capa multiproveedor.

6. Mejoras recomendadas (priorizadas)

- **Unificar datos en Prisma (D1/D2)** — base de todo.
- **Fabricante/Catálogo configurable no-code (D5)** — desbloquea el modelo de negocio.
- **Reglas de precio por fabricante** editables desde admin.
- **Cotizador determinista** — el agente con mejor retorno.
- **Bandeja omnicanal** — una sola pantalla para toda la atención.
- **Panel de configuración de agentes** más completo (reglas, plantillas, límites de gasto).
- **Observabilidad de costos de IA** (ya hay estimación; falta tablero histórico).

7. Funcionalidades faltantes (respecto a la visión multi-fabricante)

- Alta/gestión de **fabricantes** y sus catálogos.
- **Reglas de precio** por fabricante/canal (B2B vs B2C) configurables.
- **Cotizador** B2B (presupuesto → PDF → seguimiento).
- **CRM** mínimo de clientes B2B (estado, historial, próxima acción).
- **Bandeja omnicanal** (WhatsApp + Instagram/Meta).
- **BOM/costeo de producción** (para personalizados reales).
- **Compras/reposición** de insumos.
- **Dashboard de Dirección** consolidado (KPIs, no solo briefing de texto).

8. Priorización (impacto × retorno ÷ complejidad)

Iniciativa	Impacto	Retorno	Complejidad	Prioridad
F0.1 Unificar datos	Alto	Indirecto (habilitador)	Alta	1
F0.2 Fabricante/Catálogo	Alto	Alto	Media	2
A.1 Cotizador	Alto	Muy alto	Media	3
A.2 Inteligencia Comercial	Medio	Alto	Media	4
B.1 Omnicanal	Medio	Medio	Media	5
B.2 Marketing/calendario	Medio	Medio	Baja	6
C.1 Taller/BOM	Alto (con volumen)	Bajo (hoy)	Alta	7
C.2 Compras	Medio (con volumen)	Bajo (hoy)	Media	8

9. Estimación de dificultad

- **F0.1 Unificar datos:** difícil (toca todo). Es la única pieza que recomiendo hacer con cuidado quirúrgico, por entidad.
- **F0.2 Fabricante/Catálogo:** media. Entidades nuevas + pantallas de admin; el motor ya soporta config en DB.
- **A.1 Cotizador:** media. La lógica es determinista (catálogo + reglas); la IA solo redacta.
- **A.2 Inteligencia Comercial:** media. Ya existe scraper + "Mi posición"; falta empaquetarlo como agente con alertas.
- **B.1 Omnicanal:** media. WhatsApp ya está; sumar Instagram/Meta y unificar bandeja.
- **B.2 Marketing/calendario:** baja. Base ya construida.
- **C.1 Taller/BOM y C.2 Compras:** alta/media, pero **no urgentes**: rinden con volumen de producción real.

10. Recomendación técnica por mejora

- **Datos (F0.1):** Prisma como **fuentes únicas de verdad**. Migrar tablas crudas a modelos + migraciones versionadas. Eliminar `$queryRawUnsafe` salvo casos de solo-lectura muy puntuales. Migrar por entidad con doble escritura temporal.

- **Fabricante/Catálogo (F0.2):** entidades `Fabricante` , `Catalogo` , `ReglaPrecio` . Config declarativa en DB, editable desde admin. Nada hardcodeado por producto.
- **Cotizador (A.1):** motor determinista (catálogo × cantidad × reglas × margen piso 15%). IA **solo** para el texto del mensaje. Salida a PDF reutilizando el pipeline de manuales.
- **Inteligencia Comercial (A.2):** empaquetar el scraper actual como servicio "best-effort" con caché; el agente lee de la base, no scrapea en vivo en cada corrida.
- **Omnicanal (B.1):** abstraer "canal" (WhatsApp/Instagram) detrás de una interfaz común; una sola tabla de conversaciones.
- **Agentes:** mantener el motor actual. Cambiar **cuáles** están encendidos y con qué reglas, no el motor. IA siempre último recurso; por defecto modo asistido con cola de aprobaciones.
- **Costos IA:** sumar tablero histórico sobre la estimación que ya existe. Poner límite de gasto por agente configurable.
- **Vercel:** seguir en Hobby; trabajos largos por cron/lotos. Pasar a Pro solo si el Cotizador o el scraping lo exigen.

Mi propuesta (donde difiero de la idea original)

Tu instinto de "armar todos los agentes" es entendible, pero **el orden importa más que la cantidad**. Propongo:

- **No sumar agentes nuevos todavía.** Primero datos limpios (F0.1) y catálogo configurable (F0.2).
- **Encender agentes por retorno, no por completitud.** Cotizador primero (toca plata directa), Taller/BOM último (rinde con volumen).
- **La IA sigue siendo último recurso.** Todo lo que se puede hacer con código determinista, se hace con código — así ahorras tokens, que es algo que ya pediste.

Con poco tráfico, la ventaja no está en tener 8 agentes: está en tener **1 dominio de datos sólido + catálogo configurable + 1 cotizador que cierra ventas B2B**. Eso genera el volumen que después sí justifica el resto.

Estado de avance (actualizado)

Ya implementado y desplegado en la rama de trabajo:

- ☒ **Fase 0.1 — Datos unificados:** módulo canónico `src/lib/db/schema.ts` (`ensureSchema`), sin DDL disperso.
 - ☒ **Fase 0.2 — Fabricantes configurables:** alta/reglas de precio desde admin + asociación producto ↔ fabricante.
 - ☒ **Fase A.1 — Cotizador:** motor determinista + PDF + historial con estados + mensaje de envío (IA solo redacta).
 - ☒ **Fase A.2 — Inteligencia Comercial:** alertas de precio (caro/barato/competidor bajó) + agente dedicado.
 - ☒ **Fase A.3 — Research MercadoLibre (ganadores):** implementado como **best-effort**; sujeto a la fragilidad del scraping de ML.
 - ☒ **Fase B.2 — Marketing/Calendario:** calendario comercial con anticipación + generación de campañas on-demand.
 - ☐ **Fase B.1 — Atención omnicanal:** pendiente.
-

Fase D — Marketplace de agentes (visión futura, NO ahora)

Idea: convertir la Empresa IA en un **producto alquilable**: que otros comercios/empresas contraten los agentes (Cotizador, Inteligencia, Comercial...) ya "entrenados" con experiencia del rubro. Es un salto de "herramienta propia" a "SaaS multi-cliente" — es una fase lejana. **No se construye ahora**; solo se **siembran bases** que son casi gratis hoy y carísimas de agregar después.

Las 4 bases de diseño a respetar desde ya

- **Disciplina multi-tenant.** Ya existe `tenant_id` en varias tablas (hoy siempre `"default"`). Regla: **toda tabla y consulta nueva lleva `tenant_id`**, aunque hoy no se use. Sembrarlo ahora es gratis; agregarlo luego implica reescribir todas las queries.
- **Separar conocimiento del rubro (compartible) de datos del negocio (privado).** La memoria hoy mezcla todo. Etiquetar desde el inicio:
- *Compartible* (el activo alquilable): patrones del rubro, fechas fuertes, qué mensajes rinden.
- *Privado* (nunca sale del tenant): precios, clientes, proveedores, conversaciones.

Así un agente alquilado llega "sabiendo del rubro" sin filtrar datos de nadie.

- **Agentes como plantillas versionables.** Hoy son 7 fijos en código. Para alquilarlos deben ser *catálogo* instanciable por cliente con su config. Seguir empujando: **todo del agente configurable/datos, nada hardcodeado.**
- **Medir experiencia y resultados.** Ya hay telemetría (`agent_runs`) y confianza en memoria. No perder ese historial: para alquilar un agente hay que **demostrar que funciona y mejora** ("propuso X, se aprobó Y, generó Z").

Camino recomendado (orden)

- Que los agentes propios **se coordinen** (orquestración interna) y **acumulen experiencia real** con el negocio.
- Cuando un agente **demuestre resultados**, recién ahí hay algo demostrable para alquilar.
- El marketplace es **Fase D**, después de B y C. Antes, solo respetar las 4 bases en todo módulo nuevo.

Decisión abierta

¿Se alquilan **agentes sueltos** (ej. "contratá el Cotizador") o la **Empresa IA completa** como paquete por rubro?

Recomendación: diseñar pensando en **agentes sueltos** (más granular y vendible), que además se pueden empaquetar. No requiere decidirlo hoy.

Caso concreto de vertical: estudio jurídico (agente de intake/triage)

Ejemplo de cómo el marketplace se aplica a un rubro distinto a mates, reutilizando ~70% de la plataforma:

- **Qué hace el agente:** atiende consultas 24/7 (WhatsApp/web), **orienta en general** (NO da asesoramiento legal), **califica el caso** (área: laboral/familia/penal/sucesiones, urgencia, si es cliente real), toma los datos y **deriva al abogado correcto**, avisándole al estudio con el caso ya resumido y clasificado. Valor: el estudio deja de perder consultas y le llega el caso filtrado.
- **Qué se reutiliza (la plataforma):** multi-tenant (`tenant_id` por estudio), motor de agentes + herramientas + memoria, omnicanal, jefe, aprobaciones, config sin código.
- **La capa vertical (lo nuevo, ~30%):** base de conocimiento legal (áreas, FAQs, criterios de triage), reglas de derivación (qué caso → qué abogado), formulario de intake, integración con agenda/CRM del estudio.
- **Caveats (críticos en legal):** (1) nunca "asesoramiento legal", solo **orientación general + derivación**, con aclaración explícita — es tema de **responsabilidad**; (2) **confidencialidad y datos sensibles** → separación

total entre tenants; (3) el agente **no es abogado**: filtra y deriva, no ejerce.

- **Prerrequisitos: multi-tenant real** (hoy sembrado, no activo) + la capa vertical.
- **Lectura estratégica**: los mates validan y financian la plataforma; el alquiler de agentes por rubro (legal, salud, inmobiliaria, contable...) es el negocio grande de atrás, y probablemente **más escalable** que el producto físico. Un paso a la vez: primero el negocio propio funcionando, después empaquetar.

Pendiente operativo abierto (no bloquea este documento)

- **Verificación WhatsApp → Aprobaciones**: quedó por confirmar que la acción propuesta por el agente de WhatsApp aparezca en la pantalla de Aprobaciones tras el último fix (commit `fe274ed` : lista `FALLBACK` sincronizada + inserción en `actionqueue` observable). *Cuando ejecutes el agente con el código ya desplegado, avisame qué frase sale en el log ("encolado en Aprobaciones" / "no se pudo encolar..." / la vieja "proponer enviarwhatsapp")*, y cierro ese punto.
- **Token permanente de WhatsApp**: pendiente de tu lado (lo ibas a crear con el instructivo del PDF).

Esperando tu aprobación para empezar por la Fase 0. No implemento nada hasta tu OK.